

1. Selected Task

Task: Debugging Visual Studio and .NET build errors in legacy Windows Forms applications.

Version 0 — Naive Prompt

Prompt

Help me fix this Visual Studio error.

Build started...

LC0000 Could not load DevExpress...

Representative Output

The assistant identifies that a DevExpress assembly is missing and suggests reinstalling DevExpress, cleaning the solution, rebuilding, and checking project references.

Notes

Technique

None (Baseline)

Observed output

The answer identifies the error but remains generic.

Still missing

No prioritization, no explanation of why the error occurs, and no diagnostic workflow.

Next improvement

Assign the AI a specific expert role.

Version 1 — Role Assignment

Technique

Role Assignment

Prompt

You are a senior .NET developer experienced with Visual Studio, DevExpress and Windows Forms.

Help me debug this build error.

(Build log)

Representative Output

The response becomes more technical.

It explains possible causes such as

- blocked DLLs
 - missing DevExpress installation
 - incorrect project references
 - license compiler issues
-

Notes

Observed improvement

The explanation is more precise and technically accurate.

Still missing

The assistant still doesn't understand my project.

Next improvement

Provide project context.

Version 2 — Context & Motivation

Technique

Context and Motivation

Prompt

You are a senior .NET developer.

I am maintaining a legacy VB.NET Windows Forms ERP system using DevExpress 20.1.

My goal is to fix the build without upgrading libraries because other developers depend on this version.

Here is the build log:

...

Representative Output

The assistant now avoids suggesting upgrades.

It focuses on

- checking blocked DLLs
 - verifying packages
 - repairing DevExpress installation
 - matching assembly versions
-

Notes

Observed improvement

Recommendations became relevant to my actual environment.

Still missing

The troubleshooting order is unclear.

Next improvement

Specify the desired output format.

Version 3 — Output Structure

Technique

Output Structure

Prompt

You are a senior .NET developer.

Explain the problem using this structure.

1. Root Cause

2. Evidence

3. Verification Steps

4. Fix

5. Prevention

Build Log:

...

Representative Output

The assistant organizes the answer into clear sections, making it much easier to follow.

Notes

Observed improvement

The structured response is easier to execute.

Still missing

Some steps depend on previous checks.

Next improvement

Break the work into sequential tasks.

Version 4 — Step Decomposition

Technique

Step Decomposition

Prompt

You are a senior .NET developer.

Guide me through fixing this issue one step at a time.

After each step, wait for my result before continuing.

Use this structure:

Step

Reason

Expected Result

Next Step

(Build log)

Representative Output

The assistant first checks whether the DevExpress DLL exists, then verifies if Windows blocked it, followed by checking project references before suggesting repairs.

Notes

Observed improvement

The workflow becomes interactive and prevents unnecessary changes.

Still missing

The assistant has no examples of my preferred debugging style.

Next improvement

Provide an example.

Version 5 — Few-shot Examples

Technique

Few-shot Examples

Prompt

You are a senior .NET developer.

Example

Input:

"The project cannot find Newtonsoft.Json."

Expected style:

Root Cause

Verification

Commands

Fix

Now solve this problem using the same style.

(Build log)

Representative Output

The assistant mirrors the expected format consistently while maintaining technical accuracy.

Notes

Observed improvement

Responses become more consistent and require less editing.

One limitation

The few-shot example slightly increased the response length without improving technical accuracy significantly.

Claude vs ChatGPT Comparison

Aspect	Claude	ChatGPT
Tone	More explanatory and cautious	More direct and practical
Accuracy	Strong reasoning with assumptions clearly stated	Strong technical diagnosis and actionable fixes
Structure	Well organized but verbose	Concise and easy to follow
Failure Point	Sometimes overexplains	Occasionally assumes project details not explicitly provided
Best Use	Understanding complex issues	Day-to-day debugging and implementation

Final Reusable Prompt Template

You are an experienced [ROLE].

My objective is:

[GOAL]

Project Context

[TECH STACK]

Constraints

[CONSTRAINTS]

Task

[TASK]

Follow this structure:

1. Root Cause
2. Evidence
3. Verification Steps
4. Recommended Fix
5. Risks
6. Prevention

If information is missing, ask clarifying questions before making assumptions.